```
import asyncio
import logging
import os

from aiogram import Bot, Dispatcher, F
from aiogram.filters import CommandStart
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.storage.memory import MemoryStorage
from aiogram.types import CallbackQuery, InlineKeyboardButton,
InlineKeyboardMarkup, Message

from products import CATEGORIES, PRODUCTS, PROMOS

BOT_TOKEN = os.environ["BOT_TOKEN"]
ADMIN_ID = int(os.environ.get("ADMIN_ID", "0"))  # твой Telegram ID, сюда придут
заказы
SUPPORT = os.environ.get("SUPPORT_CONTACT", "@your_support")

dp = Dispatcher(storage=MemoryStorage())
BY_ID = {p["id"]: p for p in PRODUCTS}
carts: dict[int, dict[int, int]] = {}  # user_id -> {product_id: qty}


class Order(StatesGroup):
    contacts = State()


def kb(rows):
    return InlineKeyboardMarkup(
        inline_keyboard=[[InlineKeyboardButton(text=t, callback_data=d) for t, d
in row] for row in rows]
    )


def main_menu():
    return kb([
        [("🛍️ Каталог", "catalog")],
        [("🛒 Корзина", "cart")],
        [("🔥 Акции", "promo")],
        [("💬 Поддержка", "support")],
    ])
```

```
def money(n):
    return f"{n:,}".replace(",", " ") + " ₽"


async def show(c: CallbackQuery, text, markup):
    await c.message.edit_text(text, reply_markup=markup)
    await c.answer()


@dp.message(CommandStart())
async def start(m: Message, state: FSMContext):
    await state.clear()
    await m.answer("Привет! 👋 Магазин инструментов для дома, стройки и
сада.\nВыбери раздел:", reply_markup=main_menu())


@dp.callback_query(F.data == "menu")
async def menu(c: CallbackQuery, state: FSMContext):
    await state.clear()
    await show(c, "Выбери раздел:", main_menu())


@dp.callback_query(F.data == "catalog")
async def catalog(c: CallbackQuery):
    rows = [[(name, f"cat:{key}")] for key, name in CATEGORIES.items()]
    rows.append([("⬅️ Назад", "menu")])
    await show(c, "Выбери категорию:", kb(rows))


@dp.callback_query(F.data.startswith("cat:"))
async def category(c: CallbackQuery):
    key = c.data.split(":")[1]
    items = [p for p in PRODUCTS if p["cat"] == key]
    rows = [[(f'{p["name"]} — {money(p["price"])}', f'prod:{p["id"]}')] for p in
items]
    rows.append([("⬅️ Назад", "catalog")])
    text = CATEGORIES[key] if items else CATEGORIES[key] + "\n\nПока пусто."
    await show(c, text, kb(rows))


@dp.callback_query(F.data.startswith("prod:"))
async def product(c: CallbackQuery):
    p = BY_ID[int(c.data.split(":")[1])]
    text = f'{p["name"]}\n\n{p["desc"]}\n\nЦена: {money(p["price"])}'
    await show(c, text, kb([
        [("➕ В корзину", f'add:{p["id"]}')],
        [("🛒 Корзина", "cart")],
```

```
        [("⬅️ Назад", f'cat:{p["cat"]}')],
    ]))


@dp.callback_query(F.data.startswith("add:"))
async def add(c: CallbackQuery):
    pid = int(c.data.split(":")[1])
    cart = carts.setdefault(c.from_user.id, {})
    cart[pid] = cart.get(pid, 0) + 1
    await c.answer(f"Добавлено: {BY_ID[pid]['name']} (в корзине {cart[pid]}
шт.)")


def cart_text(uid):
    cart = carts.get(uid, {})
    lines, total = [], 0
    for pid, qty in cart.items():
        p = BY_ID[pid]
        s = p["price"] * qty
        total += s
        lines.append(f'• {p["name"]} × {qty} = {money(s)}')
    return lines, total


@dp.callback_query(F.data == "cart")
async def cart_view(c: CallbackQuery):
    lines, total = cart_text(c.from_user.id)
    if not lines:
        await show(c, "🛒 Корзина пуста.", kb([[("👜 В каталог", "catalog")],
[("⬅️ Назад", "menu")]]))
        return
    text = "🛒 Твоя корзина:\n\n" + "\n".join(lines) + f"\n\nИтого:
{money(total)}"
    await show(c, text, kb([
        [("✅ Оформить заказ", "checkout")],
        [("🗑️ Очистить", "clear")],
        [("👜 Добавить ещё", "catalog")],
        [("⬅️ Назад", "menu")],
    ]))


@dp.callback_query(F.data == "clear")
async def clear(c: CallbackQuery):
    carts.pop(c.from_user.id, None)
    await show(c, "Корзина очищена.", kb([[("👜 В каталог", "catalog")], [("⬅️
Назад", "menu")]]))
```

```
@dp.callback_query(F.data == "checkout")
async def checkout(c: CallbackQuery, state: FSMContext):
    if not carts.get(c.from_user.id):
        await c.answer("Корзина пуста", show_alert=True)
        return
    await state.set_state(Order.contacts)
    await show(c, "Напиши одним сообщением: имя, телефон и адрес доставки.",
kb([[("❌ Отмена", "menu")]]))


@dp.message(Order.contacts, F.text)
async def finish(m: Message, state: FSMContext, bot: Bot):
    lines, total = cart_text(m.from_user.id)
    if not lines:
        await state.clear()
        await m.answer("Корзина пуста.", reply_markup=main_menu())
        return
    u = m.from_user
    order = (
        f"🆕 Новый заказ от {u.full_name} (@{u.username or '—'}, id {u.id})\n\n"
        + "\n".join(lines)
        + f"\n\nИтого: {money(total)}\n\nКонтакты:\n{m.text}"
    )
    if ADMIN_ID:
        try:
            await bot.send_message(ADMIN_ID, order)
        except Exception:
            logging.exception("Не удалось отправить заказ админу")
    else:
        logging.warning("ADMIN_ID не задан, заказ:\n%s", order)
    carts.pop(u.id, None)
    await state.clear()
    await m.answer("✅ Заказ принят! Мы скоро свяжемся с тобой.",
reply_markup=main_menu())


@dp.callback_query(F.data == "promo")
async def promo(c: CallbackQuery):
    await show(c, PROMOS, kb([[("⬅️ Назад", "menu")]]))


@dp.callback_query(F.data == "support")
async def support(c: CallbackQuery):
    await show(c, f"💬 Вопросы по товарам и заказам: {SUPPORT}", kb([[("⬅️
Назад", "menu")]]))
```

```
async def main():
    logging.basicConfig(level=logging.INFO)
    await dp.start_polling(Bot(BOT_TOKEN))


if __name__ == "__main__":
    asyncio.run(main())
```